



RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

Lab Evaluation 2 Report

-

"Compiler Design"

Submitted by

Md Naim Parvez
Roll No: 2003015
Section: A
4th Year (Odd Semester)

Submitted to

Farjana Parvin
Assistant Professor
Department of CSE
Rajshahi University of Engineering &
Technology

Course Code: 4102

Course Title: Compiler Design Sessional

Contents

1	Introduction	1
1.1	Background	1
2	Objective	1
3	Theory	1
3.1	Lexical Analysis (Flex)	1
3.2	Syntax Analysis (Bison)	1
3.3	Flex-Bison Integration	2
4	Problem Statement	2
5	Input Data	2
5.1	Input File (input.txt)	2
6	Implementation	3
6.1	Flex File (code1.l)	3
6.2	Token Definitions	4
6.3	Bison File (code1.y)	4
6.4	Grammar Rules Explanation	5
6.5	Makefile	5
7	Execution and Results	6
7.1	Compilation Steps	6
7.2	Generated Files	6
7.3	Output Analysis (output.txt)	6
8	Detailed Analysis	7
8.1	Tokenization Process	7
8.2	Parsing Process	7
8.3	Syntax Error Analysis	7
8.4	Token-Value Associations	8
9	Key Features	8
9.1	Flex Features Demonstrated	8
9.2	Bison Features Demonstrated	8
10	Advantages and Applications	9
10.1	Advantages	9
10.2	Real-World Applications	9
11	Possible Enhancements	9
11.1	Error Recovery	9
11.2	Extended Grammar	9
11.3	Semantic Validation	9
12	Common Issues and Solutions	10

13 Conclusion	10
14 Learning Outcomes	10

1 Introduction

Compiler design is a fundamental area of computer science that bridges the gap between high-level programming languages and machine-executable code. At the heart of any compiler lies two critical phases: lexical analysis and syntax analysis. These phases transform raw source code into a structured representation that can be further processed for semantic analysis, optimization, and code generation.

1.1 Background

Lexical analysis, also known as scanning or tokenization, is the first phase of compilation. It reads the source code as a stream of characters and groups them into meaningful sequences called tokens. Each token represents a basic syntactic unit such as keywords, identifiers, operators, or literals. This process simplifies the subsequent parsing phase by reducing the complexity of character-level processing.

Syntax analysis, or parsing, follows lexical analysis and verifies that the sequence of tokens conforms to the grammatical rules of the programming language. It constructs a parse tree or abstract syntax tree (AST) that represents the hierarchical structure of the source code. This tree serves as the foundation for semantic analysis and code generation phases.

2 Objective

The objective of this laboratory exercise is to:

1. Understand lexical analysis using Flex (Fast Lexical Analyzer)
2. Implement syntax analysis using Bison (Parser Generator)
3. Tokenize and parse structured statements containing year and degree information
4. Demonstrate the integration of lexer and parser for language processing

3 Theory

3.1 Lexical Analysis (Flex)

Lexical analysis is the first phase of compilation where the source code is converted into a sequence of tokens. Flex is a tool for generating scanners (lexical analyzers) that recognize lexical patterns in text.

Key Components:

- **Tokens:** Basic building blocks (keywords, identifiers, numbers, operators)
- **Patterns:** Regular expressions that define token recognition rules
- **Actions:** C code executed when a pattern is matched

3.2 Syntax Analysis (Bison)

Syntax analysis (parsing) is the second phase of compilation where tokens are organized into a parse tree according to grammar rules. Bison is a parser generator that produces LALR(1) parsers from

context-free grammar specifications.

Key Components:

- **Grammar Rules:** Define the structure of valid statements
- **Terminal Symbols:** Tokens from the lexer
- **Non-terminal Symbols:** Abstract syntactic constructs
- **Semantic Actions:** Code executed when rules are reduced

3.3 Flex-Bison Integration

Flex and Bison work together in the compilation pipeline:

1. Flex generates `lex.yy.c` (lexical analyzer)
2. Bison generates `code1.tab.c` and `code1.tab.h` (parser and token definitions)
3. Both are compiled together to create the final parser
4. The parser reads input, tokenizes it via Flex, and validates structure via Bison

4 Problem Statement

Given statements in the format:

```
Year (2020) MBA-3  
Year (2019) MSc-2  
Year (2022) BBA-1
```

Tasks:

1. Create a Flex file (`code1.l`) to tokenize these statements
2. Create a Bison file (`code1.y`) to parse the tokenized input
3. Demonstrate successful parsing with detailed output

5 Input Data

5.1 Input File (input.txt)

```
1 Year (2020) MBA-3  
2 Year (2019) MSc-2  
3 Year (2022) BBA-1
```

Listing 1: Input statements for parsing

6 Implementation

6.1 Flex File (code1.l)

The lexical analyzer defines patterns for recognizing tokens:

```

1 %{
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include "code1.tab.h"
6 %}
7 %%
8 "Year" {
9     printf("%s -> KEYWORD\n", yytext);
10    yylval.str = strdup(yytext);
11    return KEYWORD;
12 }
13 "MBA"|"MSc"|"BBA"|"BSc"|"PhD"|"MA"|"BA" {
14     printf("%s -> DEGREE_NAME\n", yytext);
15     yylval.str = strdup(yytext);
16     return DEGREE;
17 }
18 [0-9]+ {
19     printf("%s -> NUMBER\n", yytext);
20     yylval.num = atoi(yytext);
21     return NUMBER;
22 }
23 "(" {
24     printf("(" -> LPAREN\n");
25     return LPAREN;
26 }
27 ")" {
28     printf(") -> RPAREN\n");
29     return RPAREN;
30 }
31 "-" {
32     printf("- -> DASH\n");
33     return DASH;
34 }
35 [ \t]+ { /* ignore whitespace */ }
36 \n {
37     printf("\n -> NEWLINE\n");
38     return NEWLINE;
39 }
40 . {
41     printf("%c -> UNKNOWN\n", yytext[0]);
42     return yytext[0];
43 }
44 %%
45 int yywrap() {

```

```

46     return 1;
47 }

```

Listing 2: code1.l - Flex Lexical Analyzer

6.2 Token Definitions

Pattern	Token	Description
"Year"	KEYWORD	Reserved keyword
"MBA" "MSc" "BBA"	DEGREE	Degree program names
[0-9]+	NUMBER	Numeric values
"("	LPAREN	Left parenthesis
")"	RPAREN	Right parenthesis
"_"	DASH	Hyphen/dash character
\n	NEWLINE	Line terminator
[\t]+	-	Whitespace (ignored)

Table 1: Token patterns and their recognition rules

6.3 Bison File (code1.y)

The parser defines grammar rules for statement validation:

```

1  %{
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  int yylex();
6  void yyerror(const char *s);
7  %}
8  %union {
9      char *str;
10     int num;
11 }
12 %token <str> KEYWORD DEGREE
13 %token <num> NUMBER
14 %token LPAREN RPAREN DASH NEWLINE
15 %%
16 program:
17     line_list
18     ;
19 line_list:
20     line
21     | line_list line
22     ;
23 line:
24     year_statement NEWLINE
25     | NEWLINE
26     ;
27 year_statement:

```

```

28     KEYWORD LPAREN NUMBER RPAREN DEGREE DASH NUMBER
29     {
30         printf("Parsed: Year (%d) %s-%d\n", $3, $5, $7);
31     }
32     ;
33 %%
34 int main(){
35     printf("Starting parser for year and degree information...\n");
36
37     yyparse();
38     printf("Parsing Finished\n");
39     return 0;
40 }
41 void yyerror(const char *s){
42     printf("Timestamp: %s %s\n error: %s\n", __DATE__, __TIME__, s);
43 }

```

Listing 3: code1.y - Bison Parser Specification

6.4 Grammar Rules Explanation

Production Rules:

```

program → line_list
line_list → line | line_list line
line → year_statement NEWLINE | NEWLINE
year_statement → KEYWORD LPAREN NUMBER RPAREN DEGREE DASH NUMBER

```

Rule Descriptions:

- **program:** Starting symbol, consists of one or more lines
- **line_list:** Handles multiple lines recursively
- **line:** Each line is either a year statement or empty
- **year_statement:** Matches the pattern "Year (YYYY) DEGREE-N"

6.5 Makefile

```

1 main: code1.l code1.y
2     bison -d code1.y
3     flex code1.l
4     gcc code1.tab.c lex.yy.c
5     ./a.out <input.txt> output.txt

```

Listing 4: Makefile for Building Parser

Build Process:

1. `bison -d code1.y` - Generates parser (`code1.tab.c`) and header (`code1.tab.h`)
2. `flex code1.l` - Generates lexer (`lex.yy.c`)
3. `gcc code1.tab.c lex.yy.c` - Compiles and links both components

4. ./a.out <input.txt> output.txt - Runs parser with input redirection

7 Execution and Results

7.1 Compilation Steps

```
$ make
bison -d code1.y
flex code1.l
gcc code1.tab.c lex.yy.c
./a.out <input.txt> output.txt
```

7.2 Generated Files

File	Description
lex.yy.c	Lexical analyzer source code (from Flex)
code1.tab.c	Parser source code (from Bison)
code1.tab.h	Token definitions header file
a.out	Final executable parser
output.txt	Parsing results and token trace

Table 2: Files generated during compilation

7.3 Output Analysis (output.txt)

```
1 Starting parser for year and degree information...
2 Year -> KEYWORD
3 ( -> LPAREN
4 2020 -> NUMBER
5 ) -> RPAREN
6 MBA -> DEGREE_NAME
7 - -> DASH
8 3 -> NUMBER
9 Parsed: Year (2020) MBA-3
10 \n -> NEWLINE
11 Year -> KEYWORD
12 ( -> LPAREN
13 2019 -> NUMBER
14 ) -> RPAREN
15 MSc -> DEGREE_NAME
16 - -> DASH
17 2 -> NUMBER
18 Parsed: Year (2019) MSc-2
19 \n -> NEWLINE
20 Year -> KEYWORD
21 ( -> LPAREN
22 2022 -> NUMBER
23 ) -> RPAREN
```

```

24 BBA -> DEGREE_NAME
25 - -> DASH
26 1 -> NUMBER
27 Parsed: Year (2022) BBA-1
28 Timestamp: Oct 28 2025 21:31:26
29 error: syntax error
30 Parsing Finished

```

Listing 5: Complete parsing output with tokenization

8 Detailed Analysis

8.1 Tokenization Process

For the input Year (2020) MBA-3:

1. "Year" matched as **KEYWORD**
2. "(" matched as **LPAREN**
3. "2020" matched as **NUMBER** (value: 2020)
4. ")" matched as **RPAREN**
5. "MBA" matched as **DEGREE**
6. "-" matched as **DASH**
7. "3" matched as **NUMBER** (value: 3)
8. "\n" matched as **NEWLINE**

8.2 Parsing Process

The parser recognizes the structure:

year_statement → KEYWORD LPAREN NUMBER RPAREN DEGREE DASH NUMBER

When this rule is reduced, the semantic action executes:

```
1 printf("Parsed: Year (%d) %s-%d\n", $3, $5, $7);
```

Where:

- \$3 = Second NUMBER (2020, 2019, 2022)
- \$5 = DEGREE string ("MBA", "MSc", "BBA")
- \$7 = Fourth NUMBER (3, 2, 1)

8.3 Syntax Error Analysis

The output shows a syntax error at the end:

```

Timestamp: Oct 28 2025 21:31:26
error: syntax error

```

Cause: The input file ends without a proper newline after the last statement, or there's an EOF handling issue. The parser expects a NEWLINE token but encounters EOF.

Solution: Modify the grammar to handle EOF gracefully:

```
1 line:
2     year_statement NEWLINE
3     | year_statement /* Allow statement without trailing newline */
4     | NEWLINE
5     ;
```

8.4 Token-Value Associations

The %union declaration defines how values are associated with tokens:

```
1 %union {
2     char *str;    // For KEYWORD and DEGREE tokens
3     int num;      // For NUMBER tokens
4 }
```

This allows:

- String tokens (KEYWORD, DEGREE) to carry text values
- Numeric tokens (NUMBER) to carry integer values
- Efficient semantic processing in grammar actions

9 Key Features

9.1 Flex Features Demonstrated

- **Pattern Matching:** Regular expressions for token recognition
- **Alternative Patterns:** Using | operator for multiple degree names
- **Character Classes:** [0-9]+ for numbers
- **Whitespace Handling:** Ignoring spaces and tabs
- **Token Values:** Using yylval to pass values to parser
- **Debug Output:** Printing recognized tokens for tracing

9.2 Bison Features Demonstrated

- **Context-Free Grammar:** Defining statement structure
- **Recursive Rules:** Handling multiple statements
- **Semantic Actions:** Processing matched patterns
- **Token Types:** Typed tokens with %union
- **Error Handling:** Custom error reporting via yyerror()

- **Left-to-Right Parsing:** LALR(1) parsing algorithm

10 Advantages and Applications

10.1 Advantages

1. **Modularity:** Separation of lexical and syntactic analysis
2. **Maintainability:** Easy to modify grammar rules
3. **Efficiency:** Optimized parser generation
4. **Extensibility:** Can easily add new tokens and rules
5. **Industry Standard:** Widely used in compiler construction

10.2 Real-World Applications

- Compiler front-ends (C, C++, Java)
- Configuration file parsers
- Domain-specific languages (DSL)
- Data format validators (JSON, XML parsers)
- Text processing tools
- Protocol analyzers

11 Possible Enhancements

11.1 Error Recovery

Add error productions to continue parsing after errors:

```

1 year_statement:
2     KEYWORD LPAREN NUMBER RPAREN DEGREE DASH NUMBER
3     | error NEWLINE { yyerrok; }
4     ;

```

11.2 Extended Grammar

Support additional degree formats:

```

1 "M.Tech" | "B.Tech" | "M.Phil" { return DEGREE; }

```

11.3 Semantic Validation

Add range checking for years:

```

1 year_statement:
2   KEYWORD LPAREN NUMBER RPAREN DEGREE DASH NUMBER
3   {
4       if ($3 < 1900 || $3 > 2100) {
5           fprintf(stderr, "Invalid year: %d\n", $3);
6       }
7       printf("Parsed: Year (%d) %s-%d\n", $3, $5, $7);
8   }
9   ;

```

12 Common Issues and Solutions

Issue	Solution
"undefined reference to yywrap"	Add -lfl flag or define yywrap()
"code1.tab.h not found"	Run Bison before Flex, use -d flag
Reduce/reduce conflicts	Simplify grammar, check for ambiguity
Shift/reduce conflicts	Review operator precedence, add %left/%right
Syntax error on valid input	Check for missing newlines, EOF handling
Makefile tab errors	Use actual tabs, not spaces in recipes

Table 3: Common problems and their solutions

13 Conclusion

This laboratory exercise successfully demonstrated:

1. **Lexical Analysis:** Created a Flex specification that tokenizes structured statements into keywords, numbers, symbols, and degree names
2. **Syntax Analysis:** Developed a Bison parser that validates statement structure according to defined grammar rules
3. **Integration:** Successfully integrated lexer and parser to create a complete language processor
4. **Output Generation:** Produced detailed trace output showing tokenization and parsing steps
5. **Error Handling:** Implemented basic error reporting mechanism

The parser correctly recognized and validated all three input statements, demonstrating successful pattern matching and syntax validation. The minor syntax error at EOF is a common occurrence and can be easily addressed through grammar modification.

14 Learning Outcomes

1. Understood the role of lexical analysis in compilation

2. Learned to write Flex specifications with patterns and actions
3. Gained experience with context-free grammars in Bison
4. Understood token-parser integration using `yy1val`
5. Learned to debug parsers using trace output
6. Recognized the importance of proper EOF and error handling
7. Applied compiler theory concepts to practical implementation